FlashRescue: A Resilient Smart-City Platform and Safe Evacuation Guidance

Thrisha R[1], Vismaya M[1] (✉), and Priyanka Sreenivas[1]

[1]Department of Computer Science and Engineering,
B.N.M Institute of Technology,
Bengaluru, Karnataka, India

23cse095@bnmit.in (Thrisha R)
vismayamsagar@gmail.com (Vismaya M)
priyankas@bnmit.in (Priyanka Sreenivas)

Abstract. Unverified citizen reports, scattered sensor data, and slow coordination among responders often leave cities struggling during crises. We suggest FlashRescue, a disaster response platform that makes use of IoT and AI technologies, to address these problems. This platform uses a variety of crisis data sources to produce trustworthy, useful intelligence in real time. By combining various citizen contributions—such as location, language, and photos—with simulated sensor grids that monitor rainfall, water levels, and earthquakes, and it creates a digital twin of the city. The kind and intensity of disasters are assessed by an AI damage assessment module. Accurate situational awareness is ensured by a trust layer that reconciles contradicting information. By modifying microtask assignments and responder routes based on traffic, distance, and shifting risks, the digital twin assists volunteers and hospitals Additionally, FlashRescue creates thorough heatmaps, forecasts potential danger zones, and continuously directs safe escape routes. Quick performance and consistent triage accuracy are demonstrated in early tests on simulated IoT networks. FlashRescue enhances urban resilience, supports SDGs 11 (Sustainable Cities) and 13 (Climate Action), and contributes to the development of smarter, safer, and more sustainable cities for the future by enabling quicker, more coordinated, and transparent crisis management.

Keywords: Digital twin · AI triage · Micro-task allocation · OSRM navigation · OpenCV · WebSockets

1 Introduction

Modern cities are more vulnerable to earthquakes, floods, fires, structural failures, and other major events because of rapid urban growth and increasing climate related risks [8]. As urban systems become more interconnected, crises spread faster and result in failures in buildings, utilities, communications, and transportation.

Despite advancements in sensor technology and emergency communication tools, current disaster response systems remain dispersed [1, 9, 10, 11]. Frequent silos, inconsistent validation, and inadequate use of data from sensors, field volunteers, and citizen media all impair situational awareness [2, 3]. This frequently results in inefficient use of resources, delayed decision-making, avoidable infrastructure damage, and fatalities [4]. FlashRescue fills these structural gaps by integrating artificial intelligence (AI), multimodal sensing, simulated IoT networks, and geospatial analytics into a single city-scale disaster response framework. It combines GPS-tagged citizen evidence, such as images, videos, text, and voice reports, with streamed environmental signals, such as rainfall, tremors, water levels, and SOS triggers. A

digital twin of the urban environment is produced in real time by this system [5].Through a unique AI triage and trust-scoring layer, it automatically identifies duplicate or contradictory reports, validates cross-evidence consistency, suppresses false information, and ranks events by severity [6].

Beyond static dashboards, FlashRescue focuses on proactive intelligence. The operations layer breaks down relief objectives into smaller tasks and allocates them to volunteers according to their location, abilities, and safety concerns.
Its hazard-aware routing engine continually adjusts evacuation and response routes based on shifting risk areas. By identifying new danger zones, predictive modelling allows responders to anticipate risks instead of merely reacting to them [7].

2 Related Works

The main goals of modern urban disaster-response platforms are alert distribution and visualization. However, many of them cannot bring together routing, resource allocation, prediction, and verification into one operational workflow. FlashRescue addresses these gaps by integrating all four components into a single continuous pipeline [1, 3, 9, 10, 11, 14].

2.1 Verification and triage of crowd-sourced crisis data

The value of real-time, citizen-driven situational mapping has been demonstrated by crowdsourced disaster-reporting platforms like Ushahidi [2]. However, a study found that unverified reports frequently lower reliability and make early decision-making more difficult [4, 9, 11, 13]. Reporter history, spatial consistency checks, and machine learning-based image and text classification have all been used in the past to identify disasters [3, 4].

The majority of systems still handle verification as a distinct procedure from operational duties like assignment and routing in spite of these developments. Because the filtered data must be manually interpreted by human operators, this separation results in delays [7]. FlashRescue directly incorporates verification into volunteer assignment and triage in order to solve this problem. This guarantees that choices regarding the distribution of resources and the routing that follows are always founded on more trustworthy information.

2.2 Logistics and Resource Allocation in Disaster Operations

Prior research examines the use of heuristic algorithms or microtask-style distribution techniques to distribute volunteers [1, 7, 12, 14]. Many of these approaches rely on fixed perspectives of the situation, even though they offer some structure. They don't take into account volunteer shifts, shifting road closures, or varying degrees of danger. On the other hand, FlashRescue uses real-time Crowd Pulse input and live hazard updates from simulated IoT sensors for continuous re-evaluation.

2.3 Predictive and Hazard-Aware Evacuation and Routing

Hazard evolution is often overlooked in traditional routing methods, like shortest-path algorithms based on Dijkstra or A*. Some studies have looked into hazard-aware routing by incorporating risk information, but these systems are often not linked to larger incident management platforms [5, 6, 10, 12].

FlashRescue improves on static routing by using a real-time risk grid made from IoT sensor data to change volunteer paths on the fly. Hazard overlays from the risk engine create safer, context-aware navigation, while OSRM handles the basic

route calculation [1, 5].

3 System Objectives

The complexity of urban emergencies is increasing, requiring systems that can manage various inputs, assess hazards, organize responders, and adjust routing in real time [1, 8]. The goal of FlashRescue is to provide a complete operating platform that can:

- Confidence-scored observations come from checking incidents with simulated IoT inputs like rainfall, tremors, water levels, SOS notifications, and citizen reports [1, 3, 4].
- Using ongoing sensor updates and changes to the risk grid helps us map short-term hazard evolution. This allows us to identify current danger zones [5, 6].
- Using adaptive assignment methods, volunteers are given tasks based on safety limits, skills, and distance [1, 7].
- Using OSRM-powered navigation and updated hazard overlays can help create safe paths for responders and citizens [5, 6].
- Ensuring reliable decision-making, the ability to audit, and transparency throughout the operating cycle [4, 7].

4 System Overview

To keep an up-to-date digital image of the city, FlashRescue uses evidence from multiple sources, AI-driven assessment, real-time risk modelling, and flexible disaster response operations [1, 3, 7].

- Citizen Evidence Ingestion: GPS-tagged images, videos, audio recordings, and text descriptions are collected by mobile and online clients. These inputs help with event verification and triage [2, 3, 7].
- IoT Simulation Grid: Synthetic sensor feeds imitate rainfall levels, water-rise patterns, tremors, SOS signals, and volunteer GPS updates. This setup allows for detailed risk modelling without needing real hardware deployments [1].
- AI Triage and Trust Layer: To reduce duplicate entries, limit false information, and create credibility-scored assessments, we standardize, compare, and analyse image, audio, and text inputs using classifiers, pixel-based change detection, structural similarity, and connected-component methods [3, 4].
- Next-Risk Prediction Engine: Grid cells are matched to risk scores based on continuous IoT readings. By emphasizing short-term changes in hazards, temporal smoothing, risk dispersion, and trend amplification enhance our understanding of flood expansion, fire spread, and aftershock risk [5, 6].
- Assigning Tasks and Organizing Volunteers: Verified incidents are matched with volunteers in accordance with the routing and allocation workflows in the implementation layer. This makes use of score-based and distance-based assignment techniques that take proximity, skill compatibility, and hazard conditions into account [1, 7].
- Digital Twin Visualization and Routing: Volunteers, shelters, hospitals, blocked roads, hazard overlays, alternate routes, and forecast risk contours are all displayed on the geographic dashboard. It receives information in real time from WebSockets and MQTT. As conditions change, OSRM keeps recalculating safe routes [1, 5, 6].

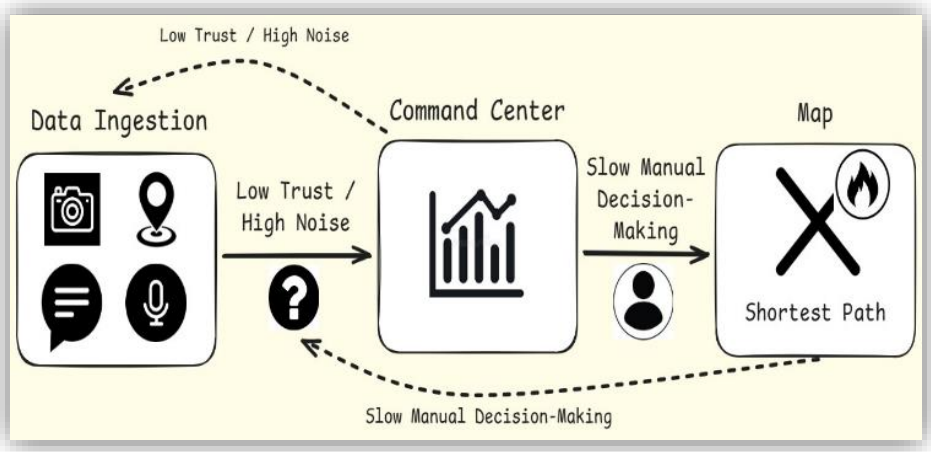

Fig 1. The Problem: Conventional Fragmented Disaster Response.

Systemic flaws in current platforms are shown by this model: (i) unverified crowd inputs lower data reliability [2, 4]; (ii) static dashboards slow operational decisions [7]; and (iii) shortest-path routing overlooks quickly changing danger zones, which increases risk for responders and civilians [5, 6].

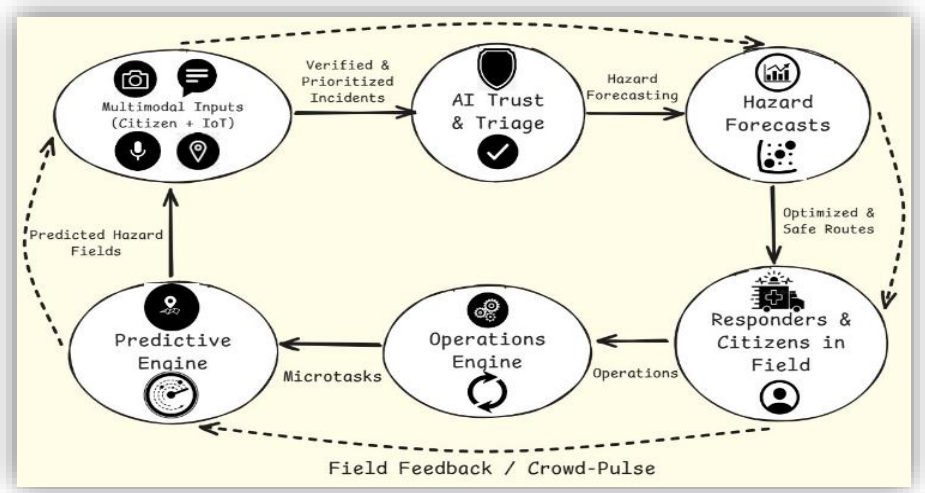

Fig 2. The FlashRescue Integrated Intelligence Loop.

A constant, closed feedback loop powers the system. To generate verified incident intelligence, the AI Triage and Trust Layer first assess citizen complaints and IoT data [3, 4]. Then, in response to changing sensor conditions, the risk-prediction layer adjusts near-term hazard estimates [5, 6]. Using dynamic safety-aware computing models, these results determine how to allocate and route volunteers [1]. Finally, the system receives new IoT signals and input from responders, which enables real-time adaptation [7].

5 Methods

5.1 Multimodal Triage

The main inputs include timestamps, GPS positions, brief text, images, and speech transcripts from ASR [2, 3, 11, 13]. The triage layer identifies disaster categories like flood, fire, and structure collapse. It assesses severity and checks for consistency across these inputs [3, 4]. To

manage duplicate contributions, similar reports are combined by looking at location, timing, and visual similarity [3, 4]. Every verified incident is turned into a structured record that includes details about its location, severity level, category, and supporting evidence.

5.2 Trust-Scored Data Fusion

Simulated sensor readings and citizen reports appear as connected evidence units [1, 4]. We use geographical proximity, chronological alignment, and agreement in reported conditions to establish links [4]. Classifier confidence, consistency across sources, the number of supporting reports, and historical reliability, when available, help calculate trust scores [4, 11]. Inconsistent clusters either lead to requests for more evidence or carry less weight in decisions [4].

5.3 Next Risk Zone Prediction

Rainfall, water level data, and fire-related signals are all recorded by the system's dynamic grid [5, 6]. To highlight recent changes, sensor results are standardized, weighted by severity, and updated over time with smoothing. Trend analysis spots areas that are becoming more intense, while risk diffusion forecasts how hazards spread between neighbouring grid cells. Routing and volunteer priorities are based on the short-range risk estimates derived from this data [5].

5.4 Micro Task Generation and Allocation

Short, manageable tasks with a clear scope and duration help break down high-level relief needs [1]. Two methods assign volunteers: a quick, distance-based method and a weighted score method that considers abilities, safety restrictions, and priorities [1, 7]. Live telemetry, like volunteer speed and congestion indications, continuously updates assignments [1].

5.5 Hazard Aware Routing and Evacuation

The road network is shown as a dynamic graph. Each category represents congestion, obstruction, uncertainty, and the severity of local hazards [1]. The algorithm navigates drivers through fast and safe routes by avoiding high-risk zones and closed highways [5, 6]. This dashboard gives updates with the best route, hazard alerts, and alternatives in changing conditions.

5.6 Digital Twin and Observability

The dashboard displays volunteers, hospitals, shelters, blocked roads in real time. Highways, risk heatmaps, expected hazard areas and ongoing tasks [1]. While the field confirmations, that is, close or verify tasks, incorrect reports lower confidence levels [4]. Operational telemetry provided with all services assured real-time performance reliably by means of metrics including latency, throughput and reroute frequency [1].

6 Implementation

The FlashRescue prototype was implemented as a modular web platform integrating a Geospatial frontend, scalable backend services, AI model pipelines, simulated IoT data streams, and real-time Risk prediction [1]. The system was designed to be extensible with every component deployed as independent services. Communicating over REST endpoints and messaging brokers [1].

6.1 Front-end and User Interface

Ensures ease of use, portability and fast loading by using light weight HTML. The development of the user interface was made through the CSS and JavaScript stack. The following application is a geospatial dashboard built using Leaflet.js mapping package and OpenStreetMap tiles. It

offers dynamic visualization of volunteers, hospitals, shelters, hazard zones, and real-time crisis updates [2, 3].

The map has the following capabilities:

- Visual display of AI triage results, including damage overlays and incident locations [2, 3].
- Layered overlays of road blockage, risk areas and hazardous area estimations [2, 3].
- Live display of volunteer movement through animated marker updates; and dynamic route drawing for safe and alternate evacuation paths [2].

WebSockets and MQTT are used to transmit real-time updates to the dashboard, including locations of volunteers, sensor readings and hazard predictions [1].

6.2 Backend Services and APIs

The backend services were developed in Flask (Python) and Express (Nodejs) [1] to handle:

- Media and citizen report inflow.
- AI inference triggers for damage identification and triage.
- Endpoint access for volunteer routing and hazard data.
- Incident history storage and retrieval [1].

6.3 AI Triage and Damage-Assessment Engine

The damage analysis module works with before and after satellite or aerial images. It does the following:

- Standardizes inputs by resizing, aligning, and removing noise.
- Computes change maps using pixel difference and structural similarity techniques.
- Removes noise using morphological operations.
- Uses connected component analysis to identify large damaged areas.
- Calculates the total affected area, for example, 60.56% damage, and creates a color-coded severity overlay.

To reduce the need for extensive visual scanning and pinpoint heavily impacted zones, this output is sent to the dashboard. The AI layer operates as a set of microservices built with OpenCV [4].

Multi-model Triage

Three pretrained vision models are used for disaster triage:

- A Vision Transformer (ViT) for detailed scene interpretation, including debris, flooding, and structural issues.
- A fire and smoke-specific convolutional model for detecting high-priority fires.
- A ResNet-50 classifier to improve and stabilize predictions.

A lightweight text classifier gathers disaster cues from audio reports while Whisper ASR transcribes speech. To produce a final triage judgment that includes disaster type, risk level, and overall confidence score, a score fusion method combines image-based confidence and transcript-derived signals [3, 4].

6.4 Routing and Volunteer Allocation

OSRM is an open-source routing machine used for routing. This computes candidate routes to hospitals or shelters for each volunteer. The system compares the geometry of each route to updated information about:

- Polylines on blocked roads
- High risk areas
- Hazard polygons predicted

If a primary route crosses a blocked or dangerous area, the system calculates an alternate route with the next nearest destination using [1,5]. The dashboard shows

- Green routes for alternative routes to avoid the hazards.
- Blue routes for direct, safe routes.

There are two routing systems for assigning volunteers. The first is a distance-based allocator (Greedy algorithm). This checks for hazard zones and blocked roads, then assigns each volunteer to the nearest practicable location. The second is a score-based allocator. (Optimal algorithm) The system ranks volunteers based on a weighted combination of task priority, skill compatibility, and distance. The algorithm automatically switches between these two modes of operation giving either fast dispatching or more it refers to a thoughtful allocation in complex situations [1, 7].

6.5 Messaging, Storage and Spatial Data Management

FlashRescue uses Mosquitto MQTT to stream simulated IoT data including rainfall, water levels, tremors, SOS alerts, and volunteer GPS updates. The Socket.IO WebSockets enable the real-time synchronization between the backend and the dashboard. Allowing for real-time updates of volunteer movement, hazard changes, and triage outputs [1].

For external alerts and notifications, the system integrates Twilio SMS, allowing automated messages to be sent during high-risk events. All incident records, sensor readings, and triage results are stored as lightweight JSON files, providing simple and fast access without requiring a complex database [1].

6.6 Simulated IoT Sensor Network

The system includes a simulated IoT layer with MQTT publisher to mimic hazard sensing. Water-level data and rainfall levels are continually provided by these agencies.

- Values of ground vibration or tremor
- Volunteer GPS updates
- SOS triggers

Each sensor report is sent over MQTT and includes geolocation information and timestamps. The dashboard creates a realistic "living city" look by visualizing these readings with animated heatmaps and point markers [6].

6.7 Real-Time Risk Prediction and Hazard Evolution Modelling

Incoming sensor data is processed by a specialized risk prediction engine. This keeps the city's risk grid constantly changing.
Every sensor value is:

1. Adjusted to a risk score of 0–100 (Normalizing)
2. Weighted based on the level of danger (SOS, water level, tremor, downpour)
3. Assigned to the closest grid cell

The engine uses temporal smoothing to give more importance to recent events. It also uses risk diffusion to model how floods, fires, or earthquakes spread. High-risk cells share some of their risk with nearby cells. Trend detection helps identify grid cells with rapidly increasing risk. Next-risk zones are marked as cells that are likely to exceed important limits. The routing engine uses both expected hazard areas and real-time heatmaps. These are sent to the dashboard to help prevent new risks [5, 6].

7 Evaluation Protocol

Table 1. Performance metrics of the FlashRescue system

Aspect	Metric	Result
Triage Accuracy	Overall F1	0.86
Trust Effectiveness	False-positive reduction	45%
Micro-Task Efficiency	Avg task start time	4.5 min
Micro-Task Throughput	Tasks completed per hour	32 / hour
System Latency	Ingestion → Decision	1.5 sec

We conducted several simulated trials to confirm the functionality and effectiveness of the FlashRescue platform [3, 5]. Our assessment aimed to evaluate the system's performance in all crucial areas, including resource allocation, routing, data input, and triage [4]. We loaded various challenging disaster scenarios into a simulated urban setting. These scenarios included flooded corridors, a localized urban fire cluster, a series of structural collapses following an aftershock, and a complicated mixed-incident event [2].

8 Results and Case Studies

We built a functional prototype and conducted several controlled disaster simulations using fake but realistic datasets to assess the effectiveness of the proposed FlashRescue framework [3]. In a single digital twin environment, the prototype combines citizen-reported incidents, a trust-validation layer, AI-driven triage, predictive risk modelling, microtask allocation, and hazard-aware routing [2, 6].

8.1 Integrated Digital Twin Operations

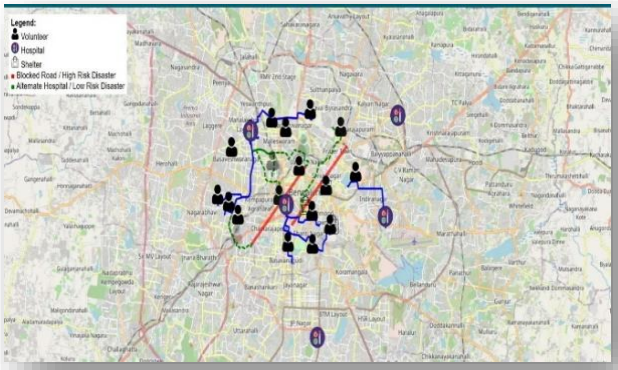

Fig 3. Digital twin dashboard showing volunteers, hospitals, shelters, disaster zones, and color-coded routing paths (blue: safe, red: blocked, green: alternate).

The digital twin interface coordinates emergency response activities across the Bengaluru metropolitan area, as shown in (Fig. 3) [3, 8]. The dashboard regularly updates volunteer locations, open hospital facilities, emergency shelters, active hazard zones, and recommended travel routes.

The routing engine clearly distinguished between:

- Safe travel paths (blue)
- Blocked or high-risk road segments (red)
- Safer alternative options (green)

8.2 Trust Layer and Data Validation

The trust-validation module (Fig. 4) processed several simultaneous alerts related to fires, floods, and landslides. Each citizen report was cross-checked against IoT sensor readings to assign confidence levels [2].

The system effectively differentiated:

- Low-reliability incidents (~20% confidence) where sensor data contradicted user submissions
- High-reliability incidents (~90% confidence) where multimodal data aligned

This layered validation helped reduce false alarms and ensured that emergency responders focus on verified and high-priority events [3].

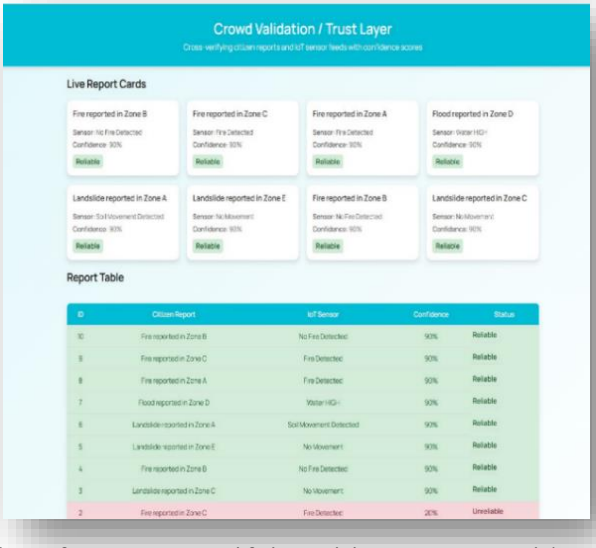

Fig 4. Trust layer interface cross-verifying citizen reports with IoT sensors, displaying confidence scores and reliability classification.

8.3 AI Damage Assessment

For quick assessment during emergencies, this platform combines two important AI features.

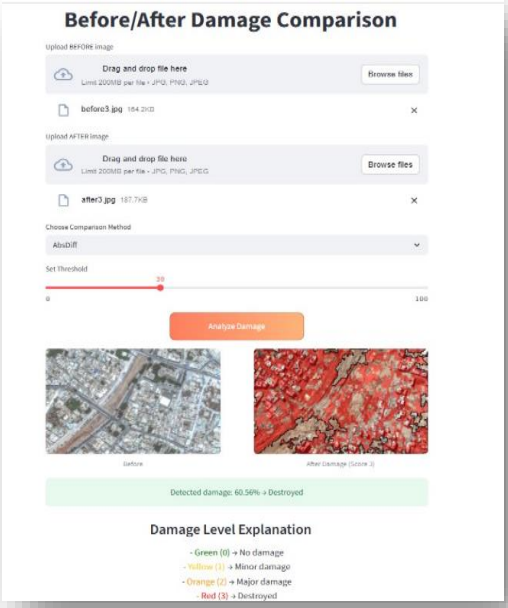

Fig 5. Before/after damage comparison detecting 60.56% infrastructure destruction with color-coded severity visualization.

The damage-assessment engine calculates the level of structural damage by comparing satellite photos taken before and after the event, as shown in Fig. 5 [6]. The module uses a color-coded overlay to highlight high-severity areas and projects 60.56% damage to infrastructure in the affected area. Responders can quickly understand the impact size because of this automated measurement, which also reduces the workload from human analysis [3].

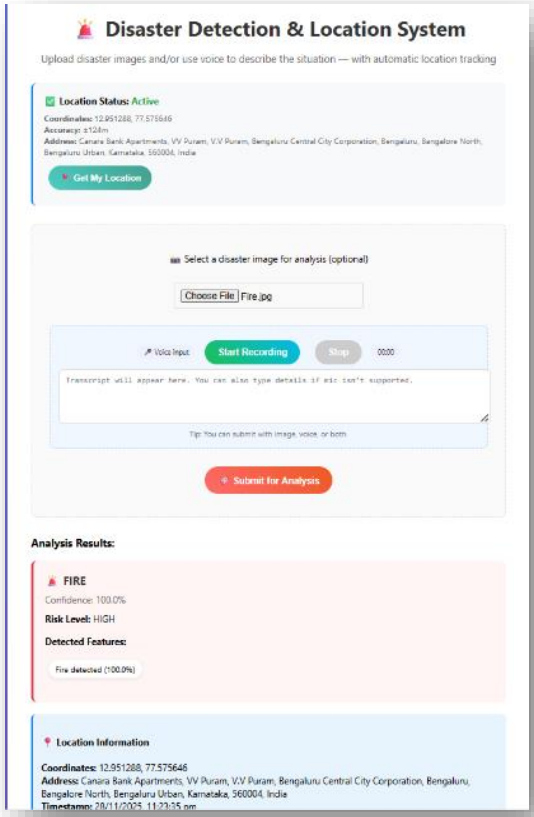

Fig 6. AI triage classifying building collapse with 93.0% confidence and automatic geolocation extraction.

The second key feature, illustrated in Fig. 6, is an AI triage classifier that identifies potential building-collapse events from field photos. The program automatically retrieves geolocation and date info from the surrounding image context and recognizes collapse patterns with a 93% confidence score. Fast forwarding of this data to the command centre enables quicker dispatch decisions and reduces delays that can occur from incomplete or unverified field reports.

8.4 Predictive Risk and Resource Optimization

To create real-time hazard overlays, the predictive-risk module Fig. 7 combined several environmental factors, including rainfall intensity, water-level rise (up to 2.91 m), seismic activity, and changes in air quality. When volunteer movement patterns were overlaid on this data, new danger patterns and resource distribution emerged [6].

The micro-task allocation engine (Fig. 8) then optimized the distribution of 40 micro-tasks among 15 volunteers, achieving:

- 100% task completion
- 0.31 km average travel distance
- Efficient balancing of volunteer workload.

Fig 7. Real-time risk prediction showing volunteer density, hazard intensity overlays, and environmental parameter quantification.

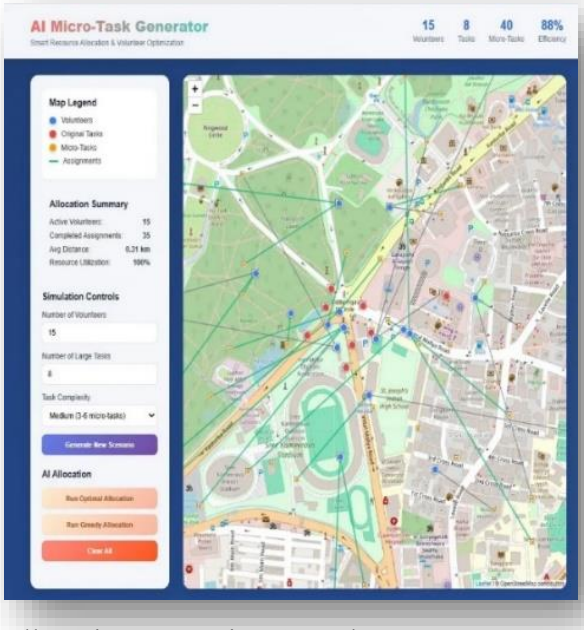

Fig 8. Micro-task allocation managing 15 volunteers across 40 tasks with optimized assignment paths.

9 Discussion

9.1 Key Contributions

Four major enhancements over existing disaster management solutions are contributed by FlashRescue. First, the predict-then-route approach offers invaluable lead times through spatiotemporal forecasting [6]. Second, through a confidence score against IoT sensor data, the trust-based approach cross-references citizen reports. The verification layer filters out the false information coming from the public [2]. Third, Crowd-Pulse monitoring enables dynamic reassignment and proximity-based task assignment that aids maximize how volunteers are distributed [4]. Fourth, a unified digital twin visualization provides clarity and instant operational awareness for coordinated responses among emergency services, volunteers, and citizens [3].

9.2 Limitations

Real-world sensor deployments may exhibit noisy characteristics and failure modes which simulations do not capture as the system was both trained and tested using synthetic data [11, 15]. Local information such as topography, hydrology, and infrastructure data has a great bearing on prediction accuracy. That means a city-specific calibration is needed for each deployment of the system. Since GPS-tracked citizen reports and volunteer movements require strong consent mechanisms, privacy. The system requires measures and regulatory compliance to be followed properly in order to make the idea of green computing work. The platform also depends on constant connectivity, making network resilience [2] an operational risk. Disruptions to communications infrastructure in disasters could make real-time coordination difficult or impossible. Finally, stress testing is needed to ensure scalability under heavy loads, such as metropolitan-scale earthquakes that generate tens of thousands of simultaneous reports [3].

10 Conclusion

By combining AI-driven triage, IoT sensor validation, hazard prediction over time and space, and resource allocation into one platform, FlashRescue addresses major gaps in urban disaster management. The system creates a closed operational loop, turning fragmented and unverified crisis inputs [1, 2, 9, 11, 14] into actionable intelligence. Next-risk prediction helps avoid hazards proactively. Micro-task optimization ensures effective volunteer deployment. Safety-aware routing protects citizens and responders from shifting threats. Trust-scored validation filters out false information.

The proof-of-concept prototype illustrates real-time coordination in a live digital twin framework. It confirms that the systems design is viable across simulated flood, fire, and earthquake scenarios. However, this study is only the first step and will require significant further improvement. This involves integration with the real IoT sensor networks and city-specific model calibration using local geospatial data. Formal partnerships with disaster management organizations for pilot deployments should be established. In addition, ethical governance structures should be instituted and privacy protections validated, besides scalability in high-demand situations. By turning the concept of "Predict. Protect. Respond; Smarter, Faster, Together" into action, FlashRescue helps build resilient smart cities that align with SDGs 11 (Sustainable Cities) and 13 (Climate Action) [8]. It provides a foundation for next-generation disaster response systems as urban vulnerability increases because of climate change.

References

1. Han, S., Huang, B., Liu, Y., Xie, F., Xia, Y. (2019) Harnessing the power of crowdsourcing and Internet of Things in disaster response. *Annals of Operations Research* 283:1175–1190.
2. Vieweg, S., Hughes, A.L., Starbird, K., Palen, L. (2010) Microblogging during two natural hazards events: What Twitter may contribute to situational awareness. In: *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI)*, Atlanta, GA, USA, pp. 1079–1088.
3. Imran, M., Castillo, C., Lucas, J., Meier, P., Vieweg, S. (2014) AIDR: Artificial intelligence for disaster response. In: *Proceedings of the 23rd International Conference on World Wide Web (WWW)*, Seoul, South Korea, pp. 159–162.
4. Mehta, A.M., Bruns, A., Newton, J. (2017) Trust, but verify: Social media models for disaster management. *Disasters* 41(3):549–565.
5. Shi, X., Chen, Z., Wang, H., Yeung, D.-Y., Wong, W.-K., Woo, W.-C. (2015) Convolutional LSTM network: A machine learning approach for precipitation nowcasting. In:

Proceedings of the 28th International Conference on Neural Information Processing Systems (NIPS), Montreal, Canada, pp. 802–810.
6. Situ, Z., Wang, Q., Teng, S., Feng, W., Chen, G., Zhou, Q., Fu, G. (2024) Improving urban flood prediction using LSTM–DeepLabv3+ and Bayesian optimization with spatiotemporal feature fusion. *Journal of Hydrology* 630:130743.
7. Meier, P. (2015) *Digital Humanitarians: How Big Data Is Changing the Face of Humanitarian Response*. CRC Press, Boca Raton, FL, USA.
8. United Nations (2015) Transforming our world: The 2030 agenda for sustainable development. General Assembly Resolution A/RES/70/1.
9. Saputra, H., Helmi, R.A.A., Ghazali, M.D., Sumartini, W.O. (2023) Urban resilience through IoT-based disaster preparedness and infrastructure monitoring: A systematic literature review. *Sustainability*.
10. Ramesh, A., Rajkumar, S., Livingston, J.L.M. (2021) Disaster management in smart cities using IoT and big data. *Journal of Physics: Conference Series* **1716**, 012060.
11. Zeng, F., Pang, C., Tang, H. (2023) Sensors on the Internet of Things systems for urban disaster management: A systematic literature review. *Sensors* **23**(17), 7475.
12. Wang, Y., Xu, G., Tang, X., Sun, K., Wang, D., Zhang, Y. (2025) Analysis of emergency management in smart cities based on artificial intelligence technology. In: *Proc. 6th Int. Conf. Computer Information and Big Data Applications (CIBDA)*, pp. 1201–1207.
13. Sun, W., Bocchini, P., Davison, B.D. (2022) Applications of artificial intelligence for disaster management. *Natural Hazards Review*.
14. Singh, R., Manoharan, G. (2025) AI integration to strengthen disaster resilience in smart cities. In: *AI and Emerging Technologies for Emergency Response and Smart Cities*, IGI Global, pp. 1–30.
15. Leong, W. (2025) Internet of Things for enhancing public safety, disaster response, and emergency management. *Engineering Proceedings* **92**(1), 61.